Public Key Infrastructure

Computer Systems Security
2025/26
LEI - UM

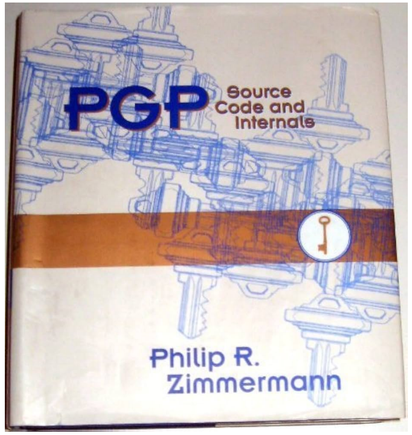
Hugo Pacheco hpacheco@di.uminho.pt

Public Key Certification

- Public key cryptography **presupposes authentic public keys**
  - Correct association between public-keys and agent identities (or MitM attacks)
- This can be solved by:
  - Trusting the public key (pre-agreed keys by assumption, no real authenticity…)
  - Assuming pre-distribution of public keys (received e.g. via a secure channel, but not far from the general key distribution problem…)
  - Relying on peer-to-peer systems such as **PGP/GPG** (decentralized web of trust)
  - Using a **Public Key Infrastructure** (centralized trust authority)

PGP

Pretty Good Privacy



- A freeware application developed by *Phil Zimmermann* (1991)
  - Privacy and authentication in data communication available to ordinary citizens
- Pioneered a legal battle in the US due to laws restricting the widespread dissemination and use of so-called strong cryptography
  - At the time, strong cryptography considered a military asset
    - Zimmermann sued for “munitions export without a license”
  - An important milestone in the balance between individual privacy and national security

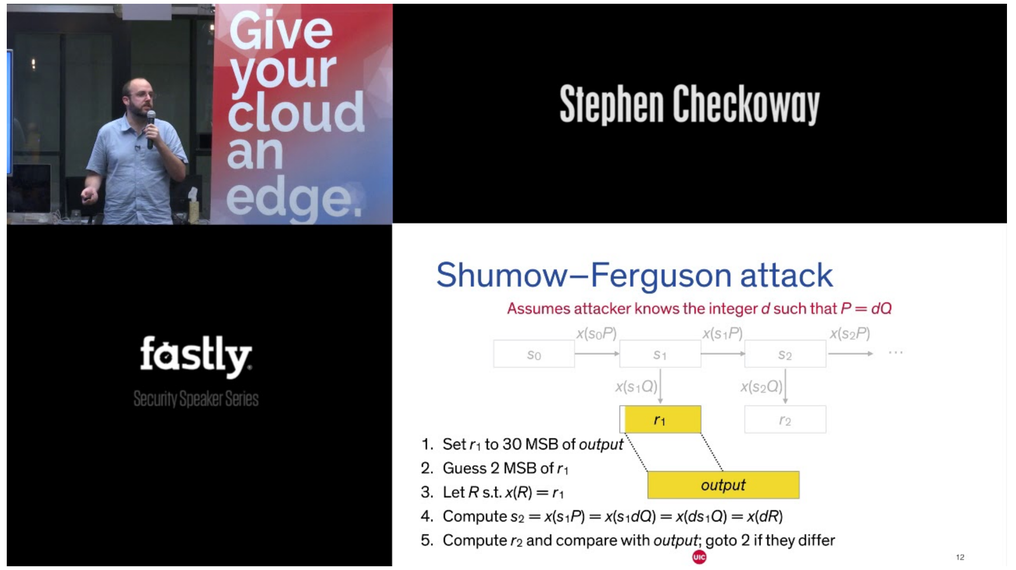
    “*if information protection is outlawed, only outlaws can have privacy!*”
- Still has applications (e.g. email) and users (e.g. journalists) today, but hard to use in practice and not too widespread

“National Security”

A settled battle?

- Not really, just less obvious

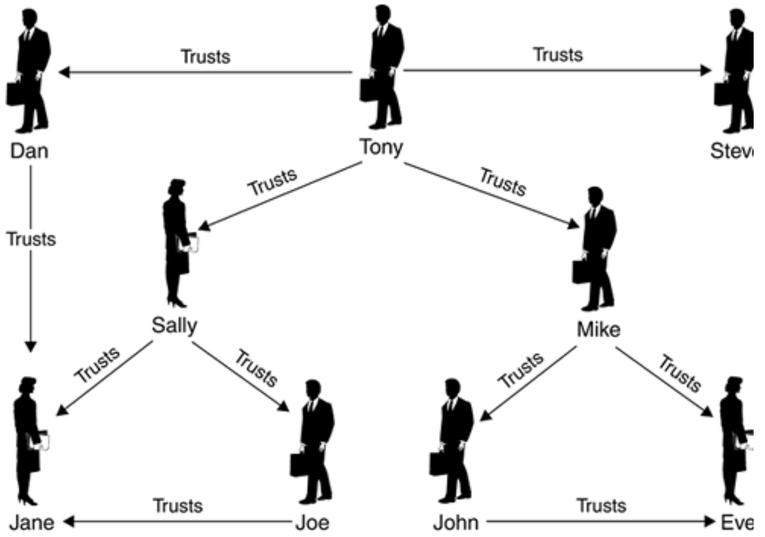
All EU member states have used surveillance spyware + need for regulation
https://www.europarl.europa.eu/doceo/document/TA-9-2023-0244_EN.html



PGP

Pretty Good Privacy

- **Web of trust** approach to public key certification
  - **Decentralized:** no single point of failure
  - **Peer-to-peer trust:** mimics the patterns of social interaction
    - Direct trust established by secure channels
    - Indirect trust via vetting by direct trustees
- PGP fingerprint
  - A short public key
  - Can be printed on business cards

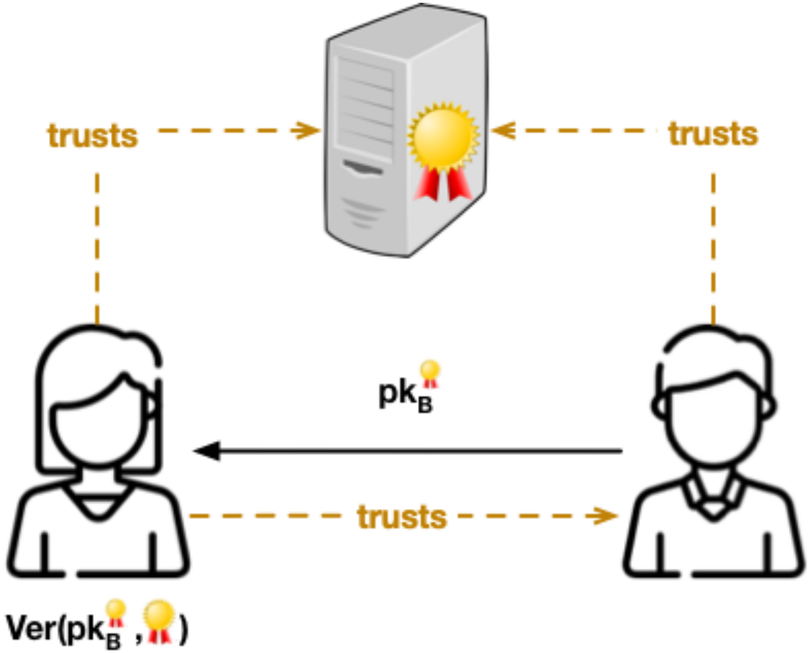


“Look at that subtle off-white coloring. The tasteful thickness of it.” (American Psycho)

PKI

Public Key Infrastructure

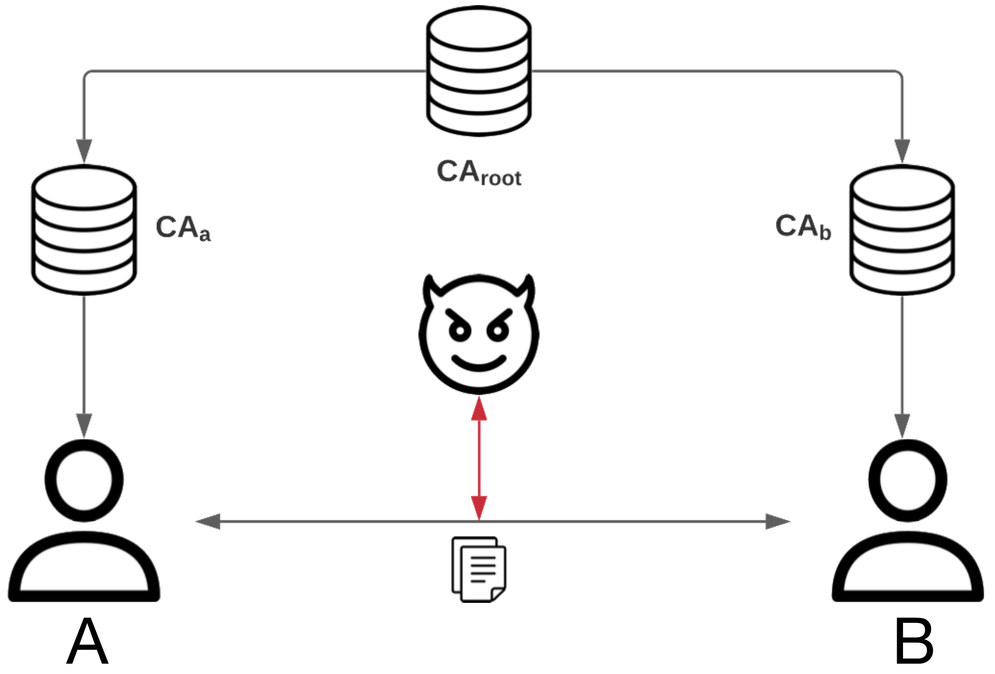
- We can use the mechanisms provided by asymmetric cryptography (specifically, digital signatures) to attest public key / identity associations
  - All agents must have the public-key of a trusted **Certification Authority** (**CA**)
    - To validate CA-signed certificates
  - The CA digitally signs each agent's public key / identity association — known as a **digital certificate**
    - Certificates can be transmitted in open channels
    - Anyone can verify a digital certificate



Public Key Infrastructure

Certificate chains

- Agents must have the public key of the CA to verify signed certificates
- But, in practice, there may be several CAs
  - A root CA certifies other CAs
    - And provides a sub-CA certificate to agents
  - Sub-CAs certify agent public keys
    - Signed with the sub-CA public key
- This is called a **certificate chain**
  - Which leads to a recursive certificate validation process
    - A validate pk_B signed by CA_a
    - A validates pk_{CA_a} signed by CA_{root}



Public Key Infrastructure

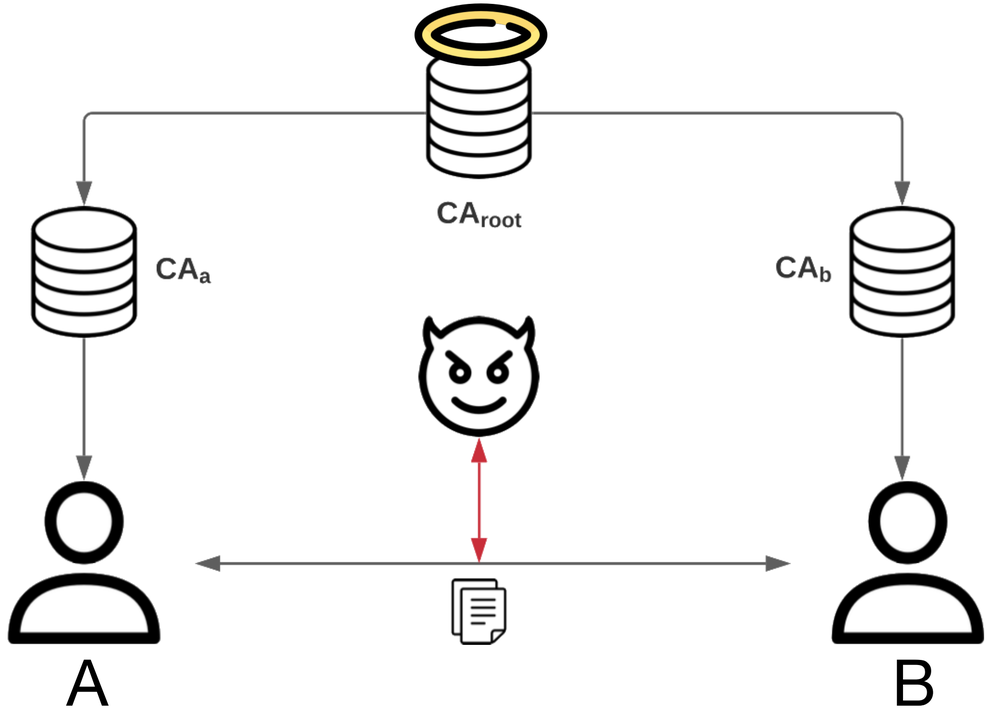
Certificate usage

- Certificate creation (e.g. for Alice):
  - Alice generates a key-pair
  - Alice requests a certificate to a CA of her choice
    - Almost never a root CA. **Why?**
  - CA carries out due diligence to ensure the correctness of Alice’s certificate. **How?**
    - Alice must sign her public key to prove that she owns her secret key
- Certificate Use:
  - When Alice needs to give Bob her public key, she sends him her certificate
  - Bob validates Alice's certificate and uses the public key it contains
- Certificates add a significant management burden to the use of public keys
  - **Only feasible for long-term key pairs**

Public Key Infrastructure

Trust hierarchy

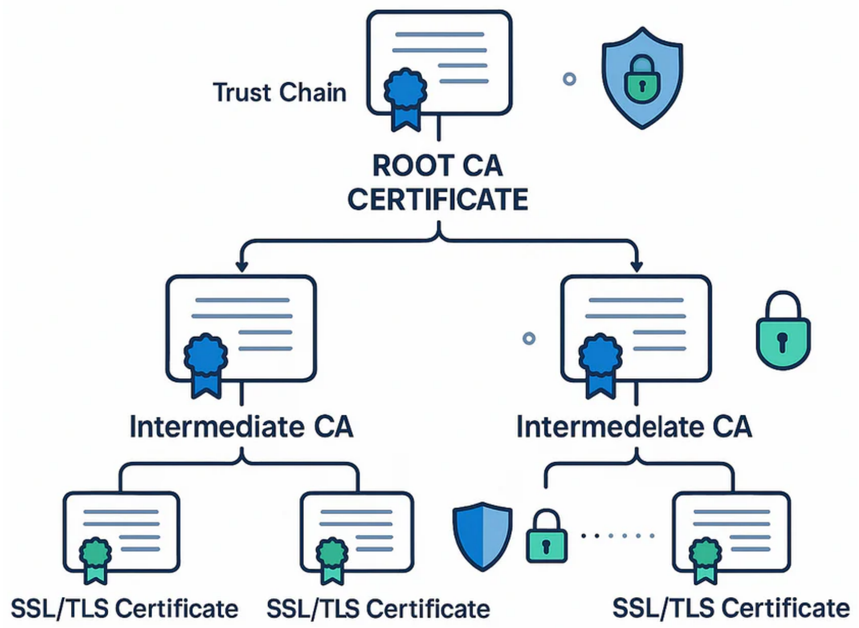
- CA_{root} is the source of trust
  - We have to assume something!
  - If CA_{root} compromised, breaks whole PKI
- Trust is hierarchical
  - It is assumed that A and B trust CA_{root}
  - But they do not need to trust CA_a or CA_b
    - Trust in sub-CAs is (recursively) anchored in the trust in CA_{root}



Public Key Infrastructure

Trusted computing base

- Root CAs = source of trust, e.g., in most secure Internet communication
- Reasonable to trust a central entity?



- Does not mean that trust is unwarranted. A lot of effort to ensure that root CAs are worthy of trust
  - Cryptographic coprocessors
  - Tamper-resistance
  - Standard-compliant APIs
  - Trusted hardware
  - “Air-gap” (offline)
  - …

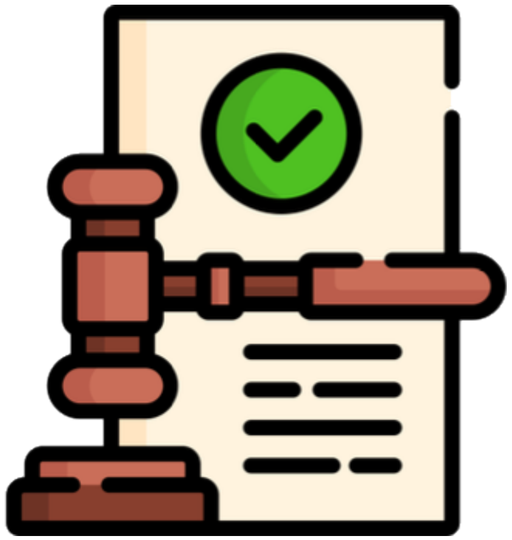
Public Key Infrastructure

Initialization

- How can an agent trust the root CA?
  - Gets the certificate via a secure channel
  - Trusts the CA’s certificate implicitly
- Some examples
  - Operating systems or browsers come with quite a lot of pre-installed CA certificates
  - National Identity Card contains authority certificates managed by the state

Public Key Infrastructure

Legal coverage & Standards



- The widespread use of public-key certificates requires a solid foundation for its establishment and systematisation
  - Technical norms: conventions, assumptions and which formats / algorithms to use
  - Practices and procedures: responsibilities and participant rights
  - Regulatory and legal framework: formal guarantees and liability on rule violation
- A PKI **bridges the gap** between
  - technological concepts (cryptographic keys)
  - ultimately social aspects (identity, individuality, personality)

Public Key Infrastructure

History

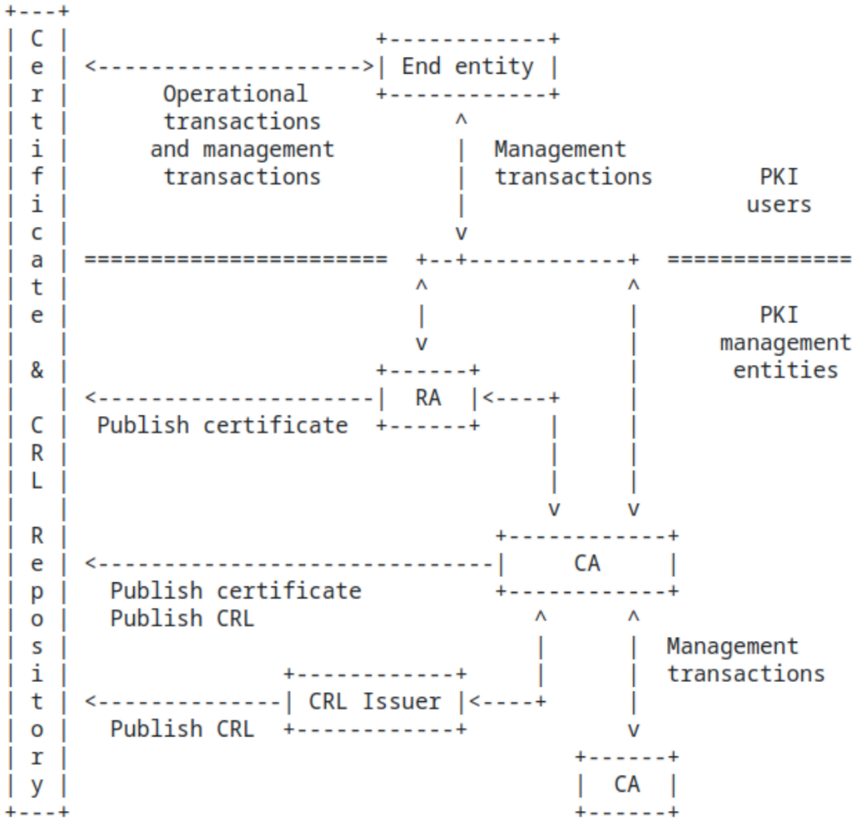
*"A **Public Key Infrastructure** (**PKI**) is a set of roles, policies, hardware, software and procedures required to create, manage, distribute, use, store and revoke **digital certificates** and manage public-key encryption."* (Wikipedia)

- An area that has largely developed in response to localised stimuli and successful solutions to specific problems that, once sufficiently mature, have been widely adopted
  - The X.509 was the first successful attempt to standardise public key certificates (ITU-1998) as the authentication mechanism for X.500 standard (Directory Services)
  - These certificates were later adopted for the SSL protocol for securing HTTP(S) connections (*Netscape* SSLv2 — 1995)
  - The IETF-PKIX-WG was set up (1995-2013) to promote the development of technical standards (RFCs) to support PKI based on X.509 certificates

Public Key Infrastructure

PKIX (PKI X.509)

- Different entities involved in a PKI (e.g. RFC 5280)
  - **Certificate holders** (**subjects**): who requests the certification of his/her public key
  - **Clients**: who need to use the public key contained in certificates
  - **Certification Authorities** (**CA**s): entities responsible in issuing/renewing/revoking certificates
  - **Registration Authorities**: (**RA**s) entities responsible in establishing the association between public keys and holders' identities (optional)
  - **Repositories**: store and make available certificates and other relevant information (such as revoked certificates, etc.)



PKIX

Operational & management transactions

- **Operational:** systems/applications to effectively rely on X509 certificates
  - In TLS, its RFC specifies how certificates can be exchanged
  - OSs/browsers manage certificates in secure components
    - This is why it is important to have a legitimate OS
  - In S/MIME, certificates are included in PKCS#7 attachments
- **Management:** how certificates are generated, distributed and stored, including their lifecycle, policies, best practices, etc
  - In public repositories (e.g. LDAP) or simply included by applications or OSes
  - Transferred by applications in specific protocols (HTTPS, FTP, MIME)
    - Must be encoded in a way that ensures interoperability

PKIX

X.509 certificates

- Technically a certificate is a document written in ASN.1
  - Abstract Syntax Notation 1: independent of platform or language
    - Represented as a dictionary data structure
    - Legacy: specification language inherited from protocol norms
  - Norms use ASN.1 to specify data structures (packages)
  - Interoperable formats = DER (binary encoding) + PEM (printable format)

```
TBSCertificate  ::=  SEQUENCE  {
    version         [0]  EXPLICIT Version DEFAULT v1,
    serialNumber         CertificateSerialNumber,
    signature            AlgorithmIdentifier,
    issuer               Name,
    validity             Validity,
    subject              Name,
    subjectPublicKeyInfo SubjectPublicKeyInfo,
    issuerUniqueID  [1]  IMPLICIT UniqueIdentifier OPTIONAL,
                         -- If present, version MUST be v2 or v3
    subjectUniqueID [2]  IMPLICIT UniqueIdentifier OPTIONAL,
                         -- If present, version MUST be v2 or v3
    extensions      [3]  EXPLICIT Extensions OPTIONAL
                         -- If present, version MUST be v3
}
```

PKIX

X.509 certificates

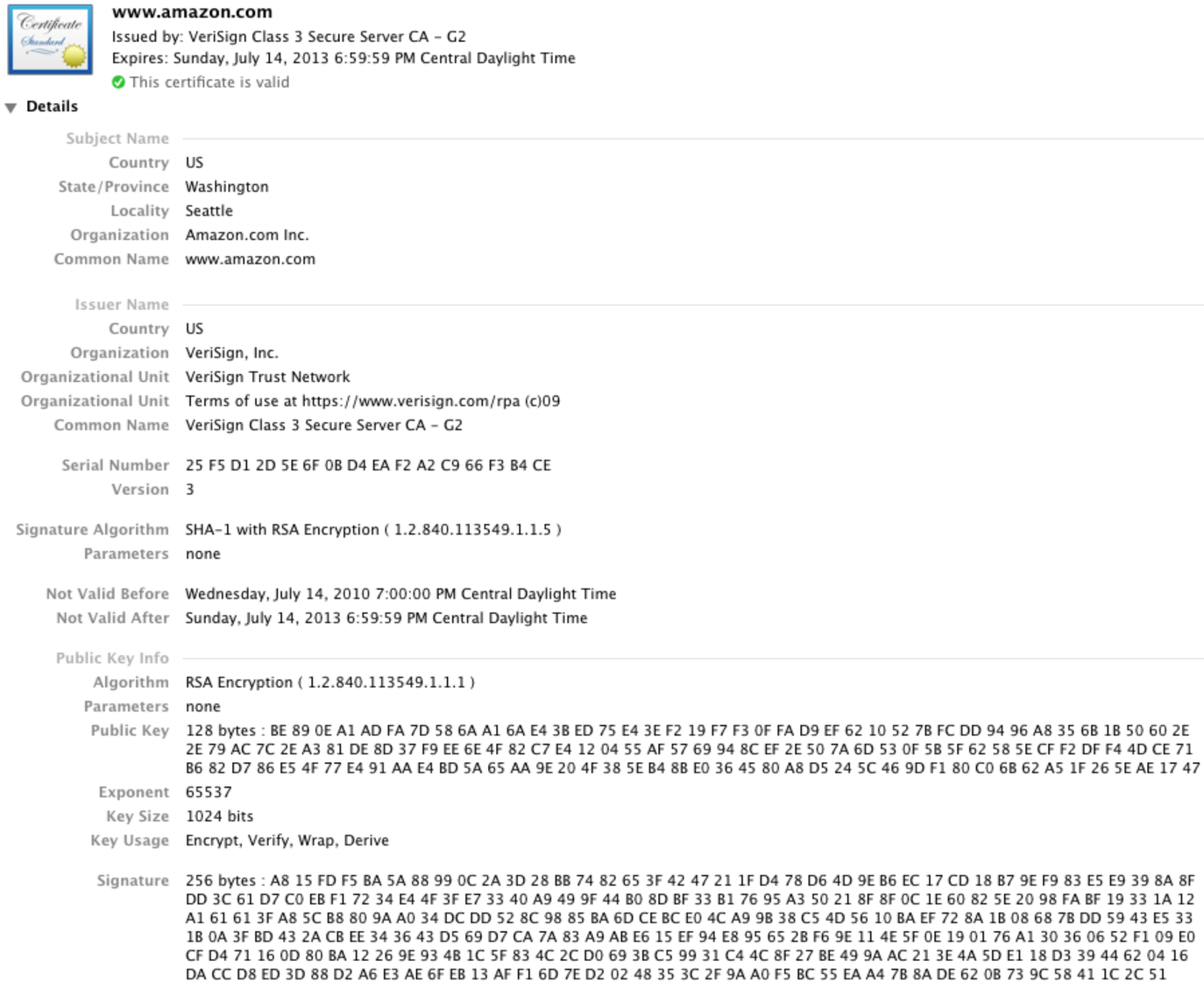
- X.509 certificates include:
  - Informative Data:
    - **subject** (i.e. who holds the private key associated with that certified public key)
    - The holder's **public key** and associated information
    - The identification of the **issuer** (signer CA)
    - Other relevant information (serial number, validity period, etc)
  - **Signature** by the CA
    - Specifically signing the DER encoding of the informative data it contains

PKIX

X.509v3 certificates

- Certificates currently in use (X509v3, 1996) addressed shortcomings of previous versions in some application domains (lack of attributes)
- Introduced an **Extensions** field, equipping certificates with the flexibility to allow generic attributes
  - Each extension is itself a data structure with an object identifier (OI) and a typed value
  - Extensions are marked as **Critical** or **Non Critical**
    - A certificate with an unrecognized critical extension must be rejected
- Important extensions
  - Basic constraints: signals if subject is a sub-CA (and optional limit on the path-length)
  - Key usage: restricts the context in which the key is to be used

A certificate example



www.amazon.com
Issued by: VeriSign Class 3 Secure Server CA - G2
Expires: Sunday, July 14, 2013 6:59:59 PM Central Daylight Time
This certificate is valid

▼ **Details**

Subject Name	
Country	US
State/Province	Washington
Locality	Seattle
Organization	Amazon.com Inc.
Common Name	www.amazon.com
Issuer Name	
Country	US
Organization	VeriSign, Inc.
Organizational Unit	VeriSign Trust Network
Organizational Unit	Terms of use at https://www.verisign.com/rpa (c)09
Common Name	VeriSign Class 3 Secure Server CA - G2
Serial Number	25 F5 D1 2D 5E 6F 0B D4 EA F2 A2 C9 66 F3 B4 CE
Version	3
Signature Algorithm	SHA-1 with RSA Encryption (1.2.840.113549.1.1.5)
Parameters	none
Not Valid Before	Wednesday, July 14, 2010 7:00:00 PM Central Daylight Time
Not Valid After	Sunday, July 14, 2013 6:59:59 PM Central Daylight Time
Public Key Info	
Algorithm	RSA Encryption (1.2.840.113549.1.1.1)
Parameters	none
Public Key	128 bytes : BE 89 0E A1 AD FA 7D 58 6A A1 6A E4 3B ED 75 E4 3E F2 19 F7 F3 0F FA D9 EF 62 10 52 7B FC DD 94 96 A8 35 6B 1B 50 60 2E 2E 79 AC 7C 2E A3 81 DE 8D 37 F9 EE 6E 4F 82 C7 E4 12 04 55 AF 57 69 94 8C EF 2E 50 7A 6D 53 0F 5B 5F 62 58 5E CF F2 DF F4 4D CE 71 B6 82 D7 86 E5 4F 77 E4 91 AA E4 BD 5A 65 AA 9E 20 4F 38 5E B4 8B E0 36 45 80 A8 D5 24 5C 46 9D F1 80 C0 6B 62 A5 1F 26 5E AE 17 47
Exponent	65537
Key Size	1024 bits
Key Usage	Encrypt, Verify, Wrap, Derive
Signature	256 bytes : A8 15 FD F5 BA 5A 88 99 0C 2A 3D 28 BB 74 82 65 3F 42 47 21 1F D4 78 D6 4D 9E B6 EC 17 CD 18 B7 9E F9 83 E5 E9 39 8A 8F DD 3C 61 D7 C0 EB F1 72 34 E4 4F 3F E7 33 40 A9 49 9F 44 B0 8D BF 33 B1 76 95 A3 50 21 8F 8F 0C 1E 60 82 5E 20 98 FA BF 19 33 1A 12 A1 61 61 3F A8 5C B8 80 9A A0 34 DC DD 52 8C 98 85 BA 6D CE BC E0 4C A9 9B 38 C5 4D 56 10 BA EF 72 8A 1B 08 68 7B DD 59 43 E5 33 1B 0A 3F BD 43 2A CB EE 34 36 43 D5 69 D7 CA 7A 83 A9 AB E6 15 EF 94 E8 95 65 2B F6 9E 11 4E 5F 0E 19 01 76 A1 30 36 06 52 F1 09 E0 CF D4 71 16 0D 80 BA 12 26 9E 93 4B 1C 5F 83 4C 2C D0 69 3B C5 99 31 C4 4C 8F 27 BE 49 9A AC 21 3E 4A 5D E1 18 D3 39 44 62 04 16 DA CC D8 ED 3D 88 D2 A6 E3 AE 6F EB 13 AF F1 6D 7E D2 02 48 35 3C 2F 9A A0 F5 BC 55 EA A4 7B 8A DE 62 0B 73 9C 58 41 1C 2C 51

PKI

Certificate binding

- CA emits certificates, but recall the **importance of binding!**
  - Signing a digital document with the subject’s information is **not enough!**
  - CA must **verify all** associated information to be included in the certificate, e.g.:
    - Subject's information such as identity and public key
    - CA’s information such as identity and PKC serial number
    - Certificate’s validity (start date and validation date)
  - Depending on the **level of assurance** (e.g., end-user vs sub-CA), this validation process can range from an automated check to a manual / legal review

PKI

Trust anchors

- A user knows a limited number of public keys belonging to CAs (generally Root CAs), and which act as the **roots of trust** (the user trusts them implicitly)
- Management of the list of trusted anchors is normally done by the OS
  - In particular, normally ensured by the OS manufacturer
- Makes the process of using certificates mostly transparent to the end user
  - **Which is good** because both the concepts and the handling of certificates is complex…
  - **Which is bad** because there is no awareness of the trust relationships involved…

PKI

Certificate validation

- By convention, **root-certificates** are **self-signed** (subject is the issuer)
- A valid certificate is typically a well-formed **certificate chain**
  - The root of the certification chain must be from a **trust anchor**
  - Recall that root CAs almost never issue user certificates
    - Which is precisely what allows them to be offline
- To validate a certificate, an entity must verify:
  - The certificate's signature
  - The applicability of the certificate (not expired, usage is according its attributes, etc)
  - If the issuer is not a trust anchor, recursively validate its certificate

PKI

Certificate revocation

- Certificates are always invalid after their validity period
- But sometimes there is a need to revoke certificates that are still valid, e.g.:
  - Private key lost or compromised
  - CAs become compromised
  - Circumstance that justified the issuer's association with the public key no longer exists (e.g. issuer is the holder of a temporary position)
  - Metadata contained in the certificate is no longer correct (e.g. attribute no longer applies)
- We can actively invalidate a certificate via **certificate revocation**

PKI

Certificate revocation

- The main mechanism is a **Certificate Revocation List** (**CRL**), a signed list of certificates revoked by a CA that should be consulted when verifying certificates
- CA periodically publishes CRLs, but how to know where the most recent CRL is?
- Three real-world solutions:
  - **Trusted Service Provider Lists (TSL)**
    - Frequently updated certificate white-list
    - Used in small/closed communities (e.g. banking)
  - **On-line Certificate Status Protocol (OCSP)**
    - Secure server checks revocation in real time
    - Used in organizational contexts (eGov)
  - **Certificate pinning**
    - Individually managed by web servers/ browsers/apps
    - Identify critical certificate revocation (e.g. Google)

Applications

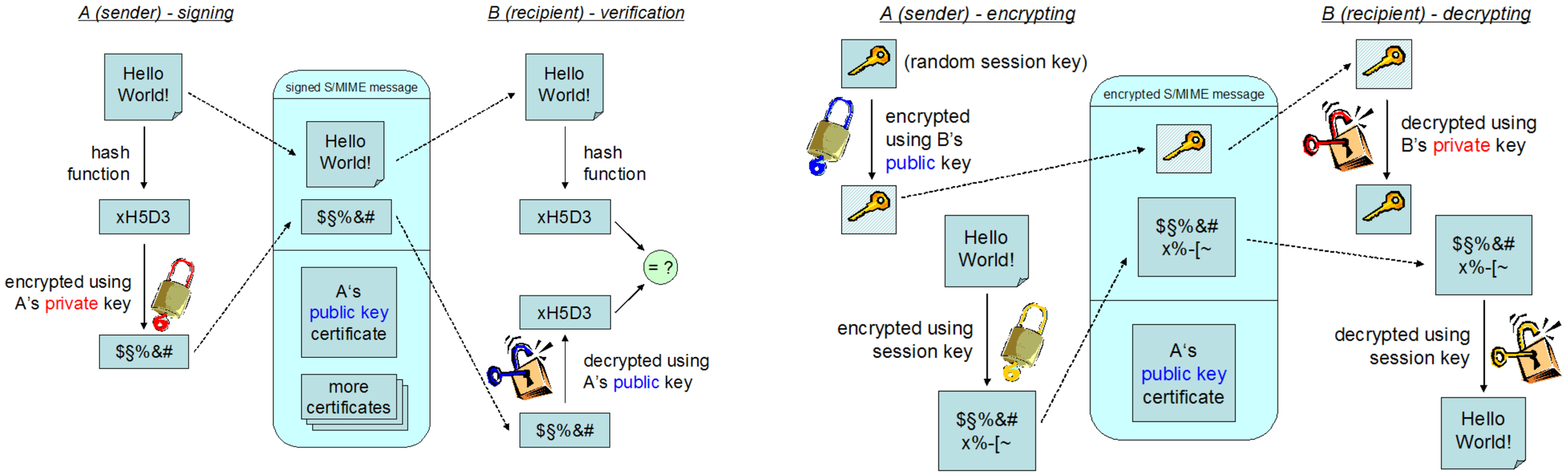
Email security

- The widespread use of email makes it a natural area to use cryptography to enhance its security
  - Remember that email is unencrypted by default!
- Early instances of secure email systems ran into considerable legal problems (specifically, PGP)
  - Making it a flagship in the promotion of online privacy
- We have already seen details about the S/MIME standard
  - Uses a **digital envelope** (hybrid PKE) to ensure **confidentiality** of messages
  - Uses **digital signatures** to enforce the **authentication**, **integrity** and **non-repudiation** of the origin of messages

Email security

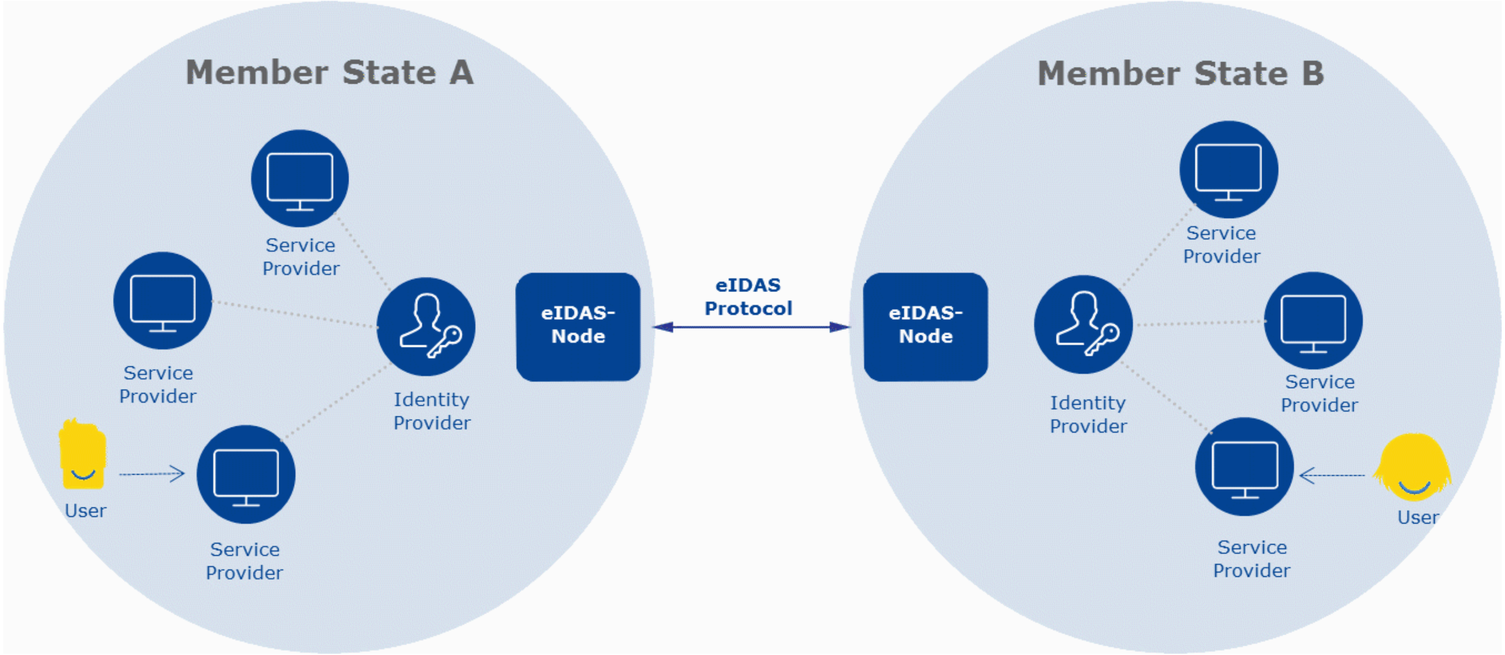
S/MIME (Secure/Multipurpose Internet Mail Extensions)

- S/MIME relies on X.509 certificates and PKIX to authenticate public keys
- Supported by most existing Email applications (Mail User Agents)



Applications

Document signing

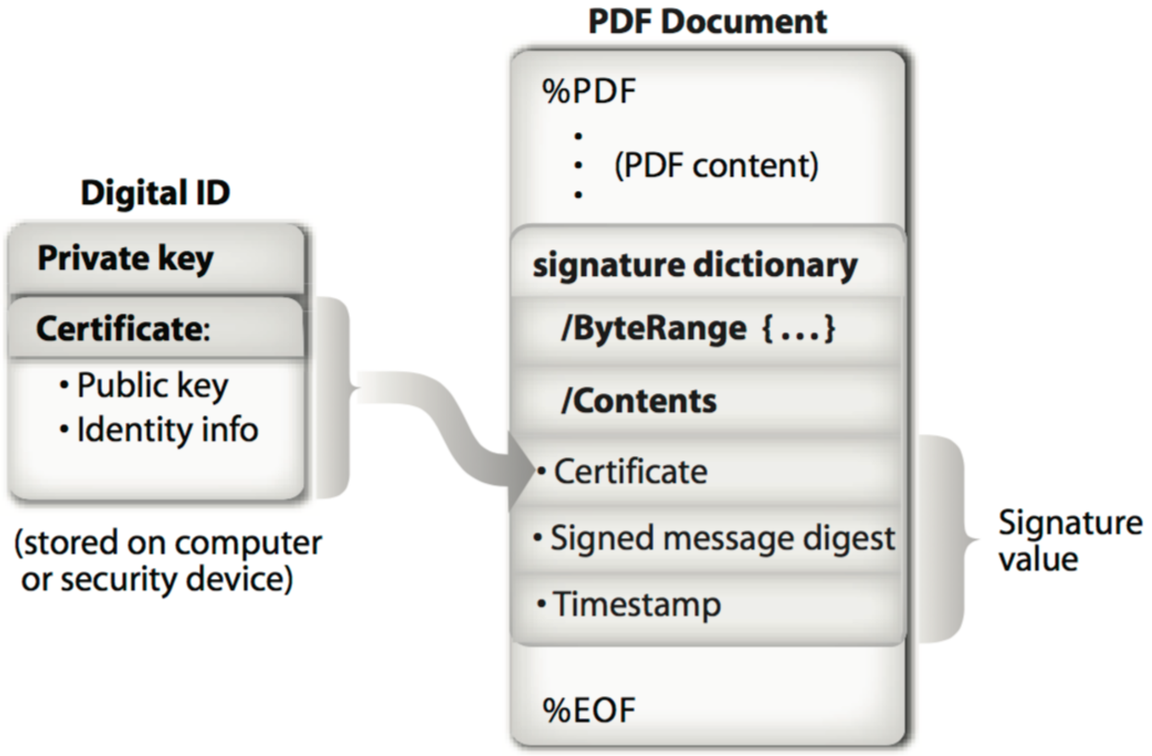


- An immediate application of digital signatures is to certify the origin of documents, which has been accompanied by legal frameworks
- A relevant large-scale recent legal effort is the EU eIDAS regulation (**e**lectronic **ID**entification, **A**uthentication and trust **S**ervices)
  - Standardize electronic transactions in the EU market
  - Interoperable and legally-binding digital signatures across EU member states
  - Create a EU-wide digital wallet to store, share, and verify digital identities, documents, and credentials

Document signing

PAdES (PDF Advanced Electronic Signatures)

- PDF is one of the most widely used formats for supporting digital documents
- Originally from a private company (Adobe), it has since been standardised by the relevant bodies (ISO 32000-2)
- PAdES is a technical standard that aligns PDF digital signatures with eIDAS
- Recent versions of Adobe Acrobat Reader support digital signatures compliant with PAdES



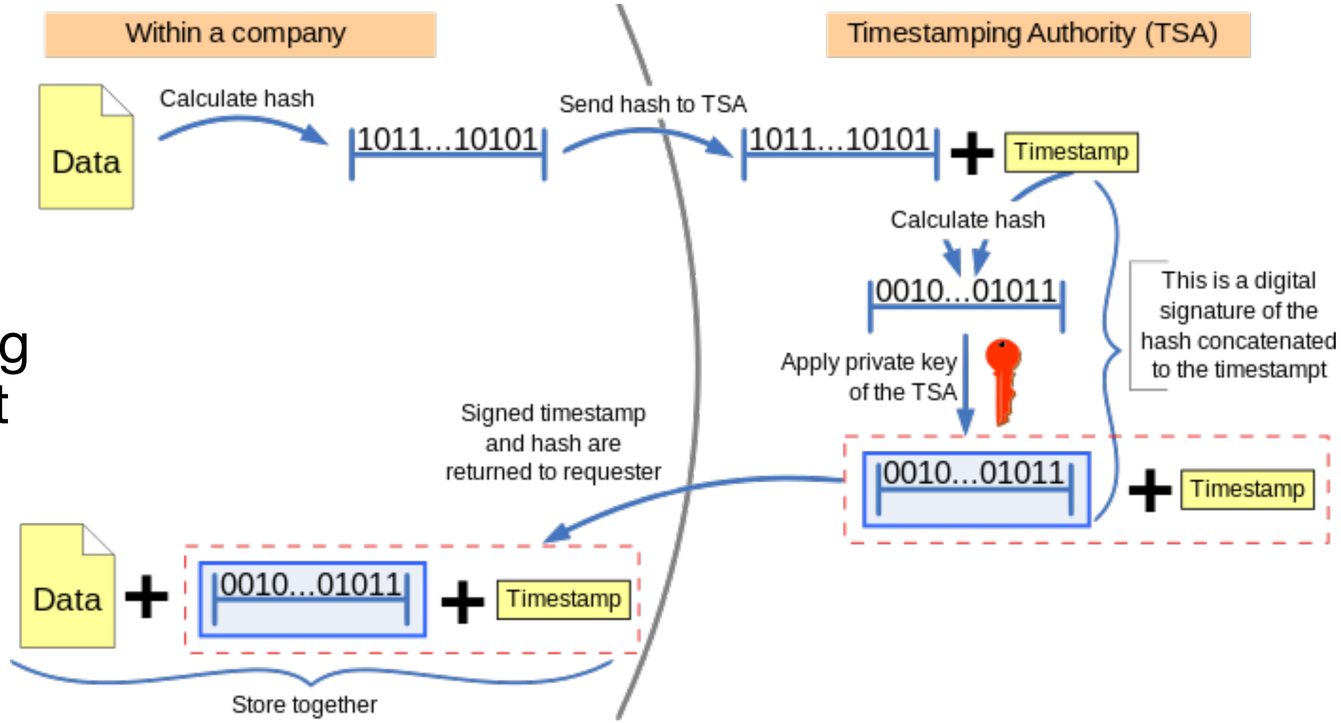
Document signing

QES (Qualified Electronic Signature)

- Qualified signatures are of particular relevance in the eIDAS context
- A **Qualified Electronic Signature** is a digital signature in which
  1. Identity of the signatory is attested by a **Qualified Certificate**
  2. Created by a secure electronic device
- Imposes strict requirements on the Certification Service Providers (CSP)
  - Each member state is responsible for accrediting CSPs with the capacity to issue qualified certificates (to be valid in all Member States)

Document signing

Trusted Timestamping



- **Timestamping** is the process of keeping track of the creation time of a document
  - Cryptographically, consists of a signature linking the content of the document and a timestamp obtained from a reliable source
  - When used in digital signatures, it establishes the time of the signature creation (important, e.g., to check the validity of the certificate at that time)
- One of the **trust services** included in the eIDAS regulation
  - A trusted timestamp is issued by a TTP acting as a **Time Stamping Authority** (**TSA**)
  - A **Qualified Timestamp** issued in the EU is valid and recognised in all Member States